DAA OUTPUT :

1) Implementation and Performance Analysis of Interpolation Search

```
Array: [2, 5, 10, 15, 23, 35, 48, 60, 75, 90, 105, 120]
Searching for: 35
Found at index: 5, Comparisons: 3

     Size    IS Time(ms)    BS Time(ms)   IS Comparisons   BS Comparisons
------------------------------------------------------------------------
     1000         0.0012         0.0014                3                9
     5000         0.0012         0.0017                3               12
    10000         0.0015         0.0016                4               11
    50000         0.0014         0.0022                4               16
   100000         0.0049         0.0037                6               12
```

2) Comparative Analysis of Naive, Rabin-Karp, and KMP Algorithms for String Matching

```
Text: AABAACAADAABAABA
Pattern: AABA

Naive -> Matches at: [0, 9, 12], Comparisons: 30
KMP -> Matches at: [0, 9, 12], Comparisons: 18
RK -> Matches at: [0, 9, 12], Comparisons: 12

     Pattern      Naive        KMP         RK
-------------------------------------------------
          AB      12470      10000       1234
        ABCD      13234      10000        212
      ABCDAB      13279      10011        159
    ABCDABCD      13281      10011        136
```

3) Implementation of Kruskal's and Prim's Algorithms for Minimum Spanning Tree

```
=== Kruskal's MST ===
Edge (0 - 3) Weight: 5
Edge (2 - 4) Weight: 5
Edge (3 - 5) Weight: 6
Edge (0 - 1) Weight: 7
Edge (1 - 4) Weight: 7
Edge (4 - 6) Weight: 9
Total MST Cost: 39

=== Prim's MST ===
Edge (0 - 3) Weight: 5
Edge (3 - 5) Weight: 6
Edge (0 - 1) Weight: 7
Edge (1 - 4) Weight: 7
Edge (4 - 2) Weight: 5
Edge (4 - 6) Weight: 9
Total MST Cost: 39
```

4) Implementation of Single Source Shortest Path Algorithm (Dijkstra's)

```
Shortest paths from vertex 0:
  Vertex   Distance                             Path
-------------------------------------------------------
       0          0                                0
       1          3                       0 -> 2 -> 1
       2          1                            0 -> 2
       3          4                  0 -> 2 -> 1 -> 3
       4          7             0 -> 2 -> 1 -> 3 -> 4
       5          9        0 -> 2 -> 1 -> 3 -> 4 -> 5
```

5) To find the MIN-Max by applying Divide and Conquer technique

```
Array: [3, 1, 7, 4, 9, 2, 8, 5, 6, 0]
Min: 0, Max: 9
D&C Comparisons: 14
Naive Comparisons: 18

    Size     DC Comps    Naive Comps   Formula 3n/2-2
-----------------------------------------------------
      10           14             18               13
     100          162            198              148
    1000         1510           1998             1498
   10000        15902          19998            14998
```